A VISUAL TUTORIAL · 20 SLIDES

pi

The minimal, extensible coding-agent harness

The agent loop, ten design decisions,
and the system prompt it actually sends.



Based on pi v1.2.0 (2026-10-12) · source read from github.com/earendil-works/pi

ORIENTATION

pi in one slide

> **“Pi is a minimal, extensible agent harness that you can make your own.”**
>
> README.md

A terminal coding agent (the `pi` CLI) plus the libraries beneath it. MIT-licensed TypeScript, Node 22.19+. Created by Mario Zechner, now developed at Earendil (npm scope `@earendil-works`).

4
default tools: read, bash, edit, write

371
words in the default system prompt

0
permission prompts before a tool call

5
front ends: TUI, print, JSON, RPC, SDK

HOW IT GOT HERE

2025-11-30
Design post: “opinionated and minimal coding agent” (then badlogic/pi-mono)

2026-04
Pi joins Earendil; the npm scope @earendil-works follows in May

2026-09-29
v0.99.0: MCP servers and codemode land

2026-10-01
v1.0.0: fullscreen TUI by default, leaner codemode

2026-10-12
v1.2.0: the release this deck is read from

Sources: README.md, CHANGELOG.md (earendil-works/pi); mariozechner.at (2025-11-30); Earendil move per eesel.ai summary (secondary).

GETTING STARTED

Install, start, steer

```
# install: pins dependencies, update with `pi update`
$ curl -fsSL https://pi.dev/install.sh | sh
# or with npm (Node 22.19 or newer)
$ npm install -g --ignore-scripts \
    @earendil-works/pi-coding-agent

# start it in the folder it should work in
$ cd /path/to/project
$ pi

> /login              subscription or API key
> /model              pick a model (Ctrl+L)
> Explain how this repository is structured
  and how to run its checks.

$ pi --continue       resume the last session here
```

WHILE PI IS WORKING

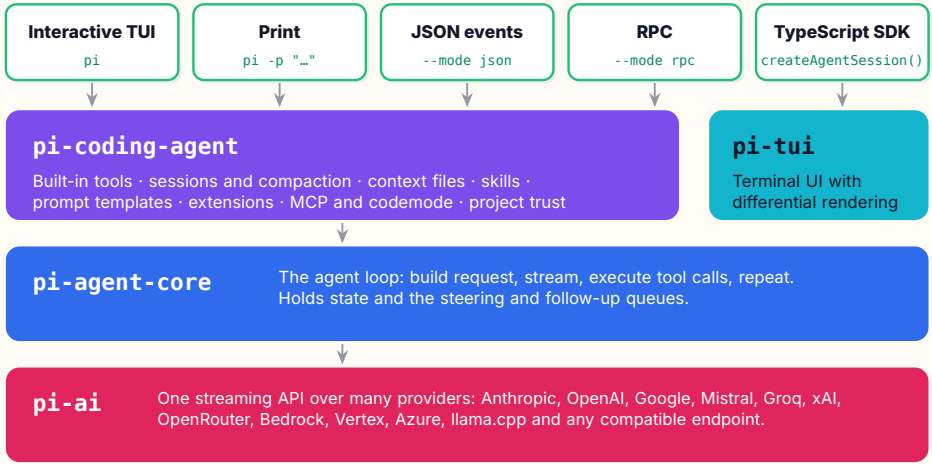
`Enter`	Steer: adjust the current task. Applied after the current turn.
`Alt+Enter`	Queue a follow-up for when the task is done.
`Esc`	Abort. Queued messages return to the editor.
`@ Tab`	Attach a file by search; complete a path.
`!cmd !!cmd`	Run a shell command; the model sees the output. `!!` hides it.
`/tree /fork`	Branch the session. `/compact` trims the history.

> “Pi shows each file read, search, command, and edit it performs. It does not ask before every tool call.”

Sources: docs/quickstart.md, docs/usage.md.

ARCHITECTURE

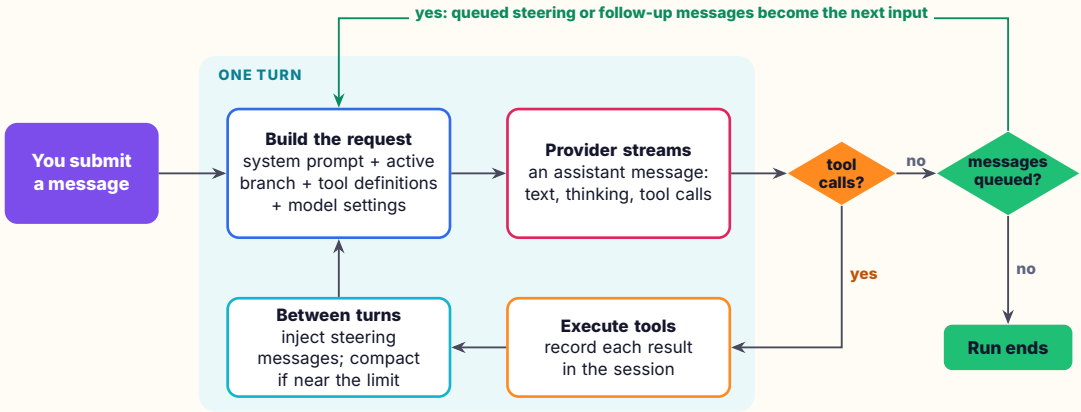
A small stack you can enter at any layer



Sources: README.md (package table), docs/how-pi-works.md: “All interfaces use the same agent and session mechanisms.”

HOW IT RUNS

The agent loop



`Enter` steer: enters after the current turn

`Alt+Enter` follow-up: waits for the run

`Esc` abort the run

Sources: docs/how-pi-works.md, docs/compaction.md, packages/agent/src/agent-loop.ts.

DESIGN DECISION 1

Trust the model: a tiny system prompt

> **“There does not appear to be a need for 10,000 tokens of system prompt”**
>
> Mario Zechner, 2025-11-30

The bet: frontier models are already trained to act as coding agents, so the harness says only what they cannot know.

What the 371 words are spent on

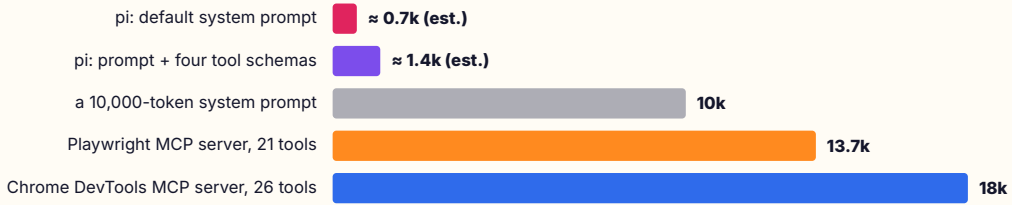
share of characters, default prompt in an empty project (2.7 KB)



identity 6% · tools · rules · pi’s own docs

grey sliver at the end: the working directory (1%)

Context spent before your first message (tokens)



est. = measured characters ÷ 4 (v1.2.0 via SDK). MCP figures: Zechner 2025, who then put pi’s prompt + tools “below 1000 tokens”.

Identity and tools

```
You are an expert coding assistant operating inside pi,
a coding agent harness. You help users by reading files,
executing commands, editing code, and writing new files.

<tools>
- read: Read file contents
- bash: Execute bash commands (ls, grep, find, etc.)
- edit: Make precise file edits with exact text replacement,
  including multiple disjoint edits in one call
- write: Create or overwrite files

In addition to the tools above, you may have access to other
  custom tools depending on the project.
</tools>
```

1. **One sentence of identity**
   Names the harness and four verbs. No persona, tone guide or safety boilerplate.

2. **One line per active tool**
   Each tool ships its own snippet. Start with `--tools read,grep,find,ls` and this list changes with it.

3. **An open door**
   Extensions, skills and MCP may add tools later. Full JSON schemas (2.7 KB for these four) travel in the request's tool list, not here.

Dumped from pi v1.2.0 with `createAgentSession().systemPrompt`: empty project, no user configuration, default tools.

The rules, and who contributes each one

`<rules>` Chip = who ships the rule. Seven of ten come from tools: drop a tool and its rules go with it.

core	- Use bash for file operations like ls, rg, find
read	- Use read to examine files instead of cat or sed.
bash	- You can inspect PI_* environment variables for current model and session details.
edit	- Use edit for precise changes (edits[].oldText must match exactly)
edit	- When changing multiple separate locations in one file, use one edit call with multiple entries in edits[] instead of multiple edit calls
edit	- Each edits[].oldText is matched against the original file, not after earlier edits are applied. Do not emit overlapping or nested edits. Merge nearby changes into one edit.
edit	- Keep edits[].oldText as small as possible while still being unique in the file. Do not pad with large unchanged regions.
write	- Use write only for new files or complete rewrites.
core	- Be concise in your responses
core	- Show file paths clearly when working with files

`</rules>`

core = added by `buildRules()` in system-prompt.ts. Rule 1 appears only when bash is active and grep, find and ls are not.

A map of its own manual, then the cwd

```
<docs>
Pi documentation (read only when the user asks about pi itself, its SDK, extensions, themes, skills, or TUI):
- Main documentation: <pi-package>/README.md
- Additional docs: <pi-package>/docs
- Examples: <pi-package>/examples (extensions, custom tools, SDK)
- When reading pi docs or examples, resolve docs/... under Additional docs and examples/... under Examples, not the
  current working directory
- When asked about: extensions (docs/extensions.md, examples/extensions/), themes (docs/themes.md), skills
  (docs/skills.md), prompt templates (docs/prompt-templates.md), TUI components (docs/tui.md), keybindings
  (docs/keybindings.md), SDK integrations (docs/sdk.md), custom providers (docs/custom-provider.md), adding models
  (docs/models.md), pi packages (docs/packages.md), environment variables (docs/environment-variables.md), MCP
  servers (docs/mcp.md), codemode scripts and non-LLM models such as classifiers and image models (docs/codemode.md)
- When working on pi topics, read the docs and examples, and follow .md cross-references before implementing
- Always read pi .md files completely and follow links to related docs (e.g., tui.md for TUI API details)
</docs>

<cwd>
/path/to/your/project
</cwd>
```

Half the prompt tells pi where its own documentation lives. That is what makes “ask Pi to build what you want” work. *Italic* = absolute path filled in at run time.

Same dump as the previous two slides. Any AGENTS.md, skills, APPEND_SYSTEM.md or MCP sections would sit between </docs> and <cwd>.

PROMPT ASSEMBLY

How the prompt is built and changed

order in the request →

`preamble`	The identity sentence. The only untagged section.
`<tools>`	One snippet per declared tool.
`<rules>`	Tool guidelines plus core rules, de-duplicated.
`<docs>`	Paths to pi's README, docs and examples.
`<addendum>`	`APPEND_SYSTEM.md`. The project file wins over the user file; they are not merged.
`<project_context>`	`AGENTS.md` or `CLAUDE.md` from `~/.pi/agent`, parent folders and the working folder.
`<skills>`	Name, description and location of each skill. The body is read on demand.
`<cwd>`	The working directory.
`<…custom>`	Sections added by extensions, for example `<mcp_servers>`.

Replace the default
A `SYSTEM.md` in `.pi/` or `~/.pi/agent/` becomes the preamble and drops the other three built-in sections.

Tagged on purpose. Each section is wrapped in a tag of its own name so a later update can be matched to it.

Patched, not rewritten. Mid-session changes are diffed per section and sent as a system-message update.

Source: buildSystemPromptSections() and diffSystemPromptSections() in src/core/system-prompt.ts; docs/configuration.md.

DESIGN DECISION 2

Four tools are enough

`read`
`(path, offset?, limit?)`

Text files and images (jpg, png, gif, webp, bmp).

Output stops at 2000 lines or 50 KB; continue with `offset`.

`bash`
`(command, timeout?)`

Runs in the working directory, returns stdout and stderr.

Keeps the last 2000 lines or 50 KB; the full output goes to a temp file.

`edit`
`(path, edits[])`

Exact text replacement, several disjoint edits per call.

Every `oldText` is matched against the original file and must be unique.

`write`
`(path, content)`

Creates or overwrites a file and its parent directories.

For new files or complete rewrites only.

Built in, but off by default

`grep` `find` `ls` `powershell` `codemode` `tool_search`

```
pi --tools read,grep,find,ls
```
a read-only reviewer

“these four tools are all you need for an effective coding agent”

Mario Zechner, 2025-11-30

Sources: src/core/tools/{read,bash,edit,write}.ts (descriptions and limits), docs/cli.md (tool selection).

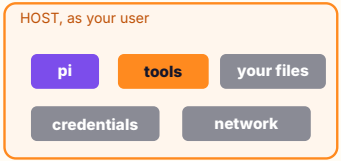
DESIGN DECISION 3

No permission prompts: isolate outside

> **“pi runs in full YOLO mode and assumes you know what you’re doing”**

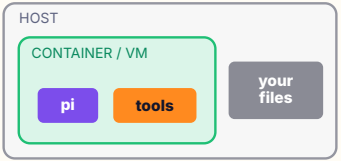
The docs are just as blunt: watching the transcript, project trust and reviewing changes “do not create a security boundary.” Limit what the process can reach.

A · Run directly



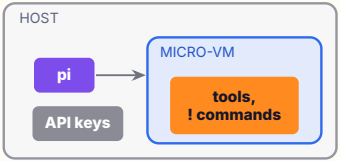
Protected: only what your OS user cannot reach anyway.

B · Whole pi in a sandbox



Docker or OpenShell: “usually the strongest practical option.”

C · Only the tools in a micro-VM



The Gondolin extension. Narrower: pi and other extensions stay outside.

Project trust is a loading gate, not a sandbox. It decides whether `.pi/` settings, MCP servers, extensions, skills and `SYSTEM.md` load. `AGENTS.md` loads either way, and tools keep the full permissions of the pi process.

Sources: docs/security.md, docs/containerization.md, README.md; quote: Zechner 2025.

DESIGN DECISION 4

What pi leaves out on purpose

“Pi ships with powerful defaults but skips features like sub-agents and plan mode. Ask Pi to build what you want, or install a package that does it your way.” README.md

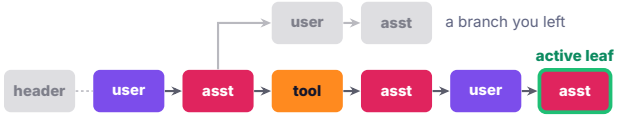
NOT BUILT IN	WHAT YOU USE INSTEAD	THE AUTHOR'S REASON
~~Sub-agents~~	A second pi you can watch: `pi -p` from bash, or a tmux pane	*“You have zero visibility into what that sub-agent does”*
~~Plan mode~~	`PLAN.md`: a file you can read, edit, diff and resume from	*Plans should be observable and outlive the session*
~~To-do tool~~	`TODO.md`, updated with ordinary edits	*“to-do lists generally confuse models more than they help”*
~~Background bash~~	tmux: long-running processes stay attachable	*“Use tmux instead”*
~~Permission prompts~~	A container, VM or sandbox (previous slide)	*Calls in-agent guardrails “mostly security theater”*

The stated principle: “if I don’t need it, it won’t be built”. MCP was on this list until v0.99.0 (slide 18). Every omission can come back as an extension or an installed package.

Sources: README.md; quotes and reasons: Zechner, “What I learned building an opinionated and minimal coding agent”, 2025-11-30.

DESIGN DECISION 5

A session is a tree in one JSONL file



Every entry stores its own `id` and a `parentId`. The model is sent the path from the root to the active leaf, and nothing else.

/tree

Jump to any entry in this file and continue from there.

/fork

New session file starting from an earlier user message.

/clone

Copy the active branch into a new session.

```
~/.pi/agent/sessions/--<path>--/
<timestamp>_<session-id>.jsonl

{"type":"session",
  "version":3, "cwd":"…"}
{"type":"message",
  "id":"a1b2c3d4",
  "parentId":"prev1234",
  "message":{"role":"user",…}}
{"type":"model_change", …}
{"type":"compaction",
  "summary":"…",
  "firstKeptEntryId":"…"}
```

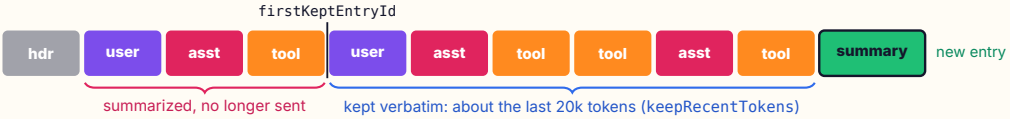
Append-only. Branching adds lines; it never rewrites history. Leaving a branch can attach a summary of it to the one you enter.

Sources: docs/sessions.md, docs/session-format.md.

DESIGN DECISION 6

Compaction summarizes; it never deletes

The session file after a compaction (entries are only ever appended)



What the model is sent on the next request



When it triggers

```
contextTokens > contextWindow - reserveTokens
```

`reserveTokens` defaults to 16,384. Checked after tools finish and before the next response, and before each new prompt. `/compact [instructions]` runs it by hand and lets you say what to preserve.

What it does

1. Walk back until about 20k recent tokens are kept.
2. Have the model summarize everything older.
3. Append a compaction entry holding the summary.
4. Rebuild context. Old entries stay in the tree.

Source: docs/compaction.md (defaults: reserveTokens 16384, keepRecentTokens 20k).

DESIGN DECISION 7

Use the weakest mechanism that works

“Start with the least powerful mechanism that meets your need”

docs/quickstart.md

more power, more code to trust

what a skill costs in the prompt until used:

```
<skill>
  <name>pdf-tools</name>
  <description>Extract text and tables from
    PDF files. Use when …</description>
  <location>…/SKILL.md</location>
</skill>
```

1 AGENTS.md

Standing instructions for a folder. Always in the prompt.

2 Prompt template

Reusable message text. Expands when you type `/name`.

3 Skill

`SKILL.md` plus scripts. Only its description is in the prompt; the body is read on demand.

4 Extension

TypeScript inside the pi process: tools, commands, event hooks, UI.

5 Pi package

Bundles all of these. Shared over npm or git: `pi install npm:…`

Sources: docs/quickstart.md (customization chooser), docs/skills.md, docs/packages.md; skill format: formatSkillsForPrompt() in src/core/skills.ts.

DESIGN DECISION 8

Extensions: add what is missing

```ts
// ~/.pi/agent/extensions/hello.ts
import type { ExtensionAPI }
  from "@earendil-works/pi-coding-agent";

export default function (pi: ExtensionAPI) {
  pi.registerCommand("hello", {
    description: "Show a greeting",
    handler: async (name, ctx) => {
      ctx.ui.notify(`Hello, ${name || "world"}!`, "info");
    },
  });
}
```

WHAT THE pi OBJECT CAN DO

`pi.on()`	hook any lifecycle event
`registerTool()`	a tool the model can call
`registerCommand()`	a new `/command`
`registerProvider()`	a model provider
`registerMcpServer()`	an MCP server
`registerVirtualModel()`	per-request model routing
`ctx.ui`	dialogs, status, rendering

EVENTS IN ONE RUN, AND WHAT A HANDLER MAY DO

`before_agent_start`	`tool_call`	`tool_result`	`message_end`	`agent_end`	`agent_before_settle`
edit prompt sections, tools, guidelines	change the input or block the call	rewrite the result	replace the final message	react to the run	append entries, ask for one more turn

Source: docs/extensions.md (example reflowed). Extensions run inside the pi process with its OS permissions; no build step needed.

DESIGN DECISION 9

MCP, revisited: put the tools behind code

2025 · NO MCP

Tool schemas are paid for up front. Playwright MCP: 21 tools, 13.7k tokens. Chrome DevTools MCP: 26 tools, 18k. That is “7-9% of your context window” before any work.

Advice then: a CLI plus a README.

2026 · MCP THROUGH CODEMODE

v0.99.0 connects MCP servers. By default their tools are registered but not declared to the model, which reaches them by writing a script.

```
pi mcp add docs --url \
  https://example.com/mcp
```



PER-SERVER EXPOSURE: HOW A TOOL REACHES THE MODEL

`codemode`	Default. Called from scripts, never declared to the model.
`deferred`	Declared only after `tool_search` finds it.
`direct`	Declared up front, like a built-in tool.
`hidden`	Registered but unreachable.

Scripts run in a QuickJS sandbox with no Node APIs, file system, network or timers. v1.0.0 then trimmed codemode’s own overhead: a GPT-5.6 request with the default tools and codemode went from about 5,300 to 3,300 tokens.

Sources: docs/mcp.md, docs/codemode.md, CHANGELOG.md (0.99.0, 1.0.0); 2025 figures: Zechner 2025.

DESIGN DECISION 10

Any model in, any front end out

Models are swappable mid-session

`/model` or `Ctrl+L` switches model, `Shift+Tab` cycles the thinking level. Changes are recorded as session entries.

Anthropic · OpenAI · Google Gemini · GitHub Copilot · Mistral · Groq · xAI · OpenRouter · NVIDIA NIM · Hugging Face · Bedrock · Vertex AI · Azure OpenAI · llama.cpp (local) · any compatible endpoint

Sign in with a subscription or an API key through `/login`. Custom endpoints go in `models.json`; extensions can add whole providers or route each request to a different model.

One agent, five ways to drive it

`pi`	Interactive terminal UI, fullscreen by default since 1.0.
`pi -p "…"`	Print mode: run the prompt, write the final answer, exit.
`pi --mode json`	The same run as a stream of JSONL events.
`pi --mode rpc`	A long-lived process: JSONL commands in, events out.

```
// 5 · the TypeScript SDK, in your own process

import { createAgentSession }
  from "@earendil-works/pi-coding-agent";
const { session } = await createAgentSession();
await session.prompt("What files are here?");
```

Sources: docs/providers.md, docs/models.md, docs/cli.md, docs/sdk.md, docs/how-pi-works.md.

RECAP

Ten decisions on one page

1 Tiny prompt

371 words. The model already knows the job.

2 Four tools

read, bash, edit, write. Bash is the universal adapter.

3 No prompts

No permission dialogs. Safety lives at the OS boundary.

4 Omissions

No sub-agents, plan mode or to-dos. Files and tmux instead.

5 Tree sessions

Append-only JSONL. Branch, fork and clone freely.

6 Compaction

Old turns are summarized. The entries stay in the file.

7 Disclosure

AGENTS.md, then template, then skill, then code.

8 Extensions

What is missing is a TypeScript file, in process.

9 MCP by script

Tools sit behind codemode, not in the prompt.

10 Open ends

Any model. TUI, print, JSON, RPC or SDK.

READ THE PRIMARY SOURCES

Code and docs: github.com/earendil-works/pi at commit 3daa202 (v1.2.0): `packages/coding-agent/src/core/system-prompt.ts`, `src/core/tools/`, `docs/`, `CHANGELOG.md`. **Rationale:** Mario Zechner, “What I learned building an opinionated and minimal coding agent”, mariozechner.at, 2025-11-30.